PROJECT BRIEF + DEMO PLAYBOOK

CloudVault

Secure file storage.
Clear collaboration.
Searchable documents.

THE 30-SECOND INTRODUCTION

CloudVault is a full-stack file workspace that lets users upload, organize, share, preview, and search their files. FastAPI enforces access and storage rules, while background workers scan files for malware and extract searchable text. A React interface brings these workflows together.

How to use this guide

Read	Pages	Purpose
Understand the project	2-5	Problem, architecture, stack, and core features.
Explain the implementation	6-10	Upload lifecycle, permissions, security, search, and data design.
Present with confidence	11-14	Demo preparation, speaking plan, questions, and future work.

Prepared for a student/team project presentation. The guide describes the current local implementation, including recent email-link, MFA, download, and sharing UI fixes.

No credentials, private environment values, user files, or personal account records are included.

02 / PURPOSE AND SCOPE

What problem does it solve?

Files are often spread across devices, folders, and chat attachments. People need one place to organize them, control who can use them, and find information inside documents without opening each file.

Project objectives

- Provide a browser-based workspace for files, folders, and recovery from Trash.
- Give senders and receivers understandable access roles and sharing visibility.
- Prevent access to newly uploaded files until malware scanning succeeds.
- Make text in PDFs and images discoverable through extraction and OCR.
- Keep long-running scan and indexing tasks outside the interactive API request.

Who uses it?

User	Typical need
Owner	Upload course notes, organize folders, share with classmates, and revoke access.
Recipient	See who shared a resource, preview/download it, or comment when permitted.
Operator	Configure storage and mail, monitor health/jobs, run migrations, and preserve backups.

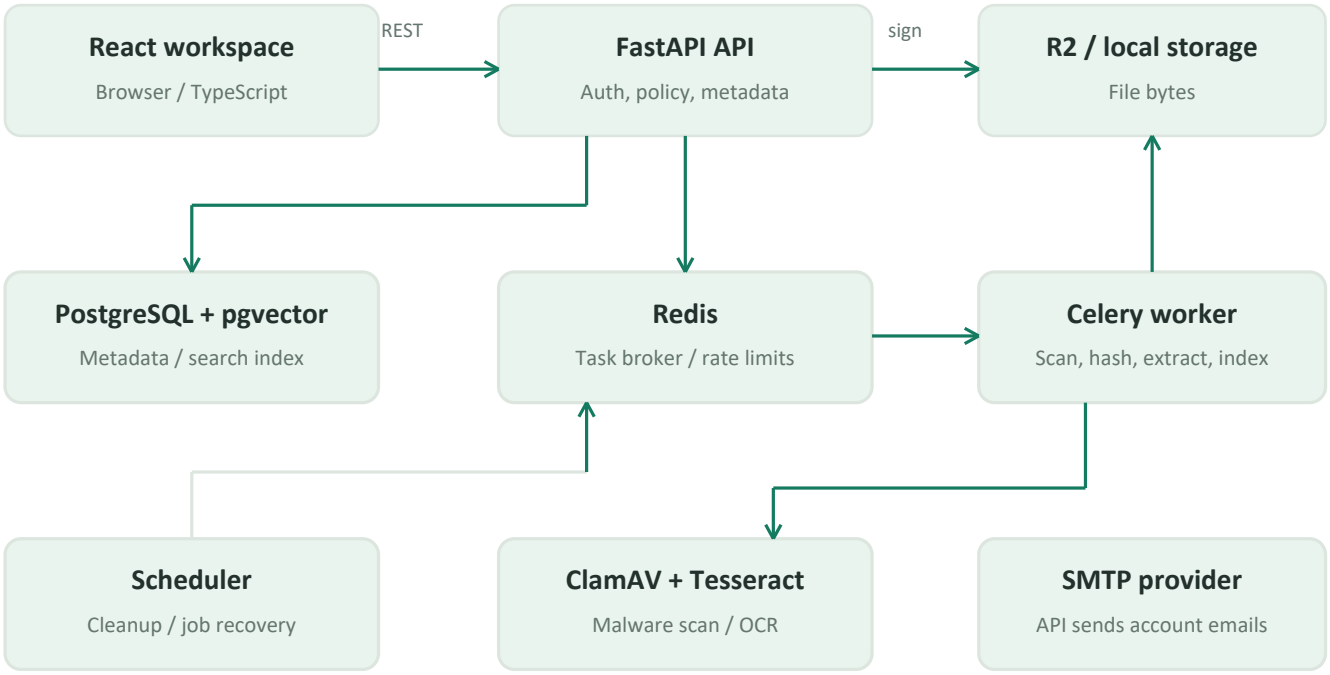
Example story

A student uploads a scanned assignment PDF. CloudVault checks the storage quota, stores the bytes, and queues a malware scan. Once clean, the file can be previewed and shared. OCR extracts its text so the owner can later find the document using words from its contents.

> **SAY THIS**
> Our focus is the complete file lifecycle: not only uploading a file, but making it safe to access, easy to organize, and clear to share.

03 / ARCHITECTURE

How the system fits together



Three paths to explain

- **Control path:** the browser asks FastAPI to authenticate users, check permissions, reserve quota, and update metadata.
- **Data path:** file bytes live in local storage or Cloudflare R2. With R2, the browser transfers bytes using narrowly scoped signed URLs.
- **Background path:** Redis queues Celery jobs; workers scan and index files. The scheduler recovers interrupted work and performs cleanup.

The browser reads owner notifications through Server-Sent Events (SSE). SMTP sends verification and password-reset links. The production template adds a Caddy frontend with HTTPS and private service networking.

DESIGN REASONING

Separate the API from expensive file processing. This keeps interactive operations responsive and allows worker capacity to grow independently.

04 / TECHNOLOGY

Tech stack and why it is used

Layer	Technology	Role in CloudVault
Frontend	React 19, TypeScript, Vite	Component-based workspace, typed API data, fast development/builds.
Interface	CSS, Lucide icons, DM Sans / Manrope	Consistent green visual style, dialogs, tables, and responsive layout.
API	Python 3.13, FastAPI, Uvicorn, Pydantic	REST endpoints, request validation, dependency-based authentication, OpenAPI docs.
Data	PostgreSQL 16, SQLAlchemy, psycopg, Alembic	Relational metadata, transactions, row locks, and schema migrations.
Search	pgvector, PostgreSQL full-text search	Ranked content results; 384-dimensional vectors.
Storage	Local filesystem or Cloudflare R2, boto3	Payload storage, signed URLs, multipart uploads.
Background	Celery, Redis, APScheduler	Scan/index jobs, rate limits, scheduled recovery and cleanup.
Document processing	ClamAV, Tesseract, pypdf, pypdfium2, Pillow	Malware scanning, PDF text extraction, scanned-page rendering, image OCR.
Account security	PyJWT, pwdlib/Argon2, PyOTP, cryptography	Tokens, password hashes, authenticator codes, encrypted MFA secrets.
Operations / tests	Docker Compose, Caddy, pytest, Playwright, GitHub Actions	Reproducible services, deployment template, backend/browser checks, CI.

KEEP THIS EXPLANATION SIMPLE

React is the user interface, FastAPI is the policy and business-logic layer, PostgreSQL stores metadata, and R2 stores the file bytes. Redis and Celery handle background work.

05 / CAPABILITIES

Core features users can demonstrate

Feature group	Present in the project
Workspace	Nested folders; rename and move; local/R2 upload; storage usage; cancel and resume supported R2 upload sessions.
File access	Download clean files; open previews from the filename or menu; PDF, plain text, PNG, JPEG, and WebP preview support.
Collaboration	Shared with me / Shared by me; sender and recipient identities; role descriptions; grant, change, and revoke access; inherited folder access.
Public links	Optional password, expiry, download count/limit, and revocation. These are separate from account permissions.
Comments and revisions	Read comments; post with Commenter/Editor access; owner-controlled replacement and restore of clean file versions.
Recovery	Move files to Trash, restore them, and permanently delete them with storage-reference cleanup.
Search	Filename/content matching, ranked snippets, PDF text extraction, image/scanned-PDF OCR, visible indexing failures and retry.
Account and activity	Login/register, email verification, password reset, authenticator MFA, recovery codes, activity history, and owner scan/index notifications.

The details that make the demo convincing

Show a pending file becoming clean, then preview it. Switch to a second account and show the same file with fewer controls. Search for a word that appears inside a document but not in its filename.

DEMONSTRATION RULE

Use a small set of prepared, non-sensitive sample files. A clean text file and a readable PDF are better than a large, unpredictable upload during a short presentation.

06 / WORKING: FILES

From upload to a downloadable file

Step	What happens	Why it matters
1. Request	User selects a file. The API validates the request and available quota.	Stops unsupported or over-quota uploads early.
2. Reserve	An R2 upload intent reserves capacity and returns a short-lived single PUT or multipart plan.	Concurrent uploads cannot freely oversubscribe quota.
3. Transfer	Browser sends bytes to R2. Local development also supports an API upload route.	R2 transfer avoids routing every byte through the API.
4. Finalize	API checks object size and digest metadata, creates file metadata, and queues scanning.	An uploaded object is not immediately trusted.
5. Scan	Celery streams bytes to ClamAV and computes a SHA-256 digest.	Downloads stay blocked until the file is CLEAN.
6. Resolve	Clean payloads can attach to a shared storage blob. Infected payloads are removed and quota released.	Deduplication reduces physical storage; malware is blocked.
7. Index	Clean files proceed to extraction/OCR and search indexing. Owner notifications report progress.	Search becomes available asynchronously.
8. Access	An authorized request resolves a short-lived R2 download/preview URL, or an authenticated local response.	API credentials are not forwarded to object storage.

Failure and recovery

Incomplete uploads can be cancelled or expire. Supported multipart sessions resume by selecting the same file in the same tab. Job leases prevent stale workers from committing newer state; a scheduler finds interrupted scan/index jobs.

IMPORTANT DISTINCTION

The client-provided digest metadata is a finalization check; the worker computes the content digest from actual bytes during scanning. Do not describe client metadata alone as proof of integrity.

07 / WORKING: ACCESS

Sharing roles that have clear meaning

Action	Viewer	Commenter	Editor	Owner
Preview / download clean files	Yes	Yes	Yes	Yes
Read comments	Yes	Yes	Yes	Yes
Post comments	No	Yes	Yes	Yes
Rename / move; move files to Trash	No	No	Yes	Yes
Create subfolders in shared folder	No	No	Yes	Yes
Upload into shared folder	No	No	No	Yes
Manage access / public links	No	No	No	Yes
Replace / restore versions; permanently delete	No	No	No	Yes

Two separate ways to share

- **Account access:** owner enters an existing username or email, selects a role, and the recipient signs in. Incoming and outgoing views show people, role, date, and file readiness.
- **Public link:** a bearer link provides download access subject to its password, expiry, active state, and download limit. It does not give editing or access-management rights.

Inheritance and revocation

Folder permissions apply to descendants when there is no closer explicit permission. A file-level grant can override inherited access. Ownership is tied to the actual owner record and cannot be granted through the role selector.

Revoking a grant stops new authorized access checks. An R2 URL already issued can remain usable until its short expiry; revocation is not retroactive deletion of a URL already in someone's possession.

ANSWER A COMMON QUESTION

Viewer includes downloading. It is a limited collaboration role, not a download-prevention or digital-rights-management feature.

08 / WORKING: SECURITY

Accounts, verification, and MFA

Sign-in and session protection

- Passwords are stored as hashes using pwdlib's recommended Argon2 scheme, not as readable passwords.
- The API uses JWT access tokens. The browser holds access/CSRF values in sessionStorage; refresh tokens use HttpOnly, SameSite cookies.
- Refresh tokens rotate. Reuse detection and authentication-version checks invalidate older sessions after relevant security events.
- Origin/CSRF checks protect cookie refresh/logout flows. Redis-backed rate limits protect authentication and public-share endpoints.

Email verification and reset

SMTP sends a single-use link valid for 30 minutes. The database stores a token digest, purpose, expiry, consumption state, and delivery status. Verification updates the account; a password reset also invalidates previous sessions. The frontend shows safe delivery diagnostics to the signed-in account.

Local email links now target the frontend. Legacy links sent to the API root redirect there. SMTP can use STARTTLS on 587 or SSL on 465; provider credentials remain private configuration.

Authenticator MFA: what the presenter should explain

Enter the current CloudVault password, request a setup key, add it as a time-based account in an authenticator app, then submit the current six-digit code. The secret is encrypted with a private Fernet key. Reused time steps are rejected, and ten hashed one-use recovery codes are issued.

The local key is persisted in a Git-ignored file. Preserve it across restarts. Email verification and authenticator MFA are separate workflows; an SMTP app password is only for the sender's mail connection.

AVOID THIS CLAIM

The project does not implement end-to-end file encryption. HTTPS, access checks, hashed passwords, and encrypted MFA secrets address different parts of security; malware scanning also cannot guarantee every threat is detected.

09 / WORKING: SEARCH

How a file becomes searchable

Stage	Implementation
Extract	Plain text is decoded; text-based PDFs use PDF text extraction; images use Tesseract OCR.
Render if needed	Scanned PDF pages are rendered with pypdfium2 and passed to Tesseract. Page, pixel, and time limits bound processing.
Represent	Filename and extracted content produce a searchable record. Vectors have 384 dimensions and record the embedding model identity.
Rank	Search combines exact phrase/name matches, PostgreSQL full-text ranking, and vector similarity. Results include snippets.
Isolate	Current content search filters to the signed-in owner's clean, non-deleted current files. It is not a global search across everyone's files.
Recover	Index status and failures are visible. The owner can retry indexing; scheduler recovery handles interrupted work.

Lexical search versus learned semantic search

The default vector is deterministic token hashing: useful for lightweight lexical similarity, but it does not inherently understand synonyms or meaning. An optional local sentence-transformers model can provide learned semantic embeddings.

Enabling that model requires installing optional dependencies, configuring the same 384-dimensional model for the API and worker, and reindexing the files. Model weights and representative relevance/performance checks are still deployment work.

Simple demo example

Upload a document named **week-03.pdf** containing the phrase **database normalization**. After scanning and indexing, search for that phrase and show the content snippet. Use a readable document: OCR accuracy depends on the source image.

> **SAY THIS ACCURATELY**
> We support content search and OCR today. Learned semantic search is an optional mode, and we distinguish it from the default lexical implementation.

10 / ENGINEERING

Data model and backend structure

Entity / group	Responsibility
users	Identity, password hash, quota counters, verification, MFA, and authentication version.
folders / files	Ownership and hierarchy; file metadata, storage location, scan/index states, soft deletion; revisions use version_of.
storage_blobs	Backend plus SHA-256 identity and references to clean physical payloads.
permissions / shared_files	Account role grants versus public-link rules, expiry, passwords, and download counters.
upload_intents	Reserved capacity, object identity, expiry, multipart state, and completion tracking.
comments / notifications / audit_logs	Discussion, owner-facing events, and recorded activity.
account_tokens / refresh_tokens / token_blocklist	Expiring account actions, rotating refresh sessions, and access-token revocation.
Search records	Extracted text, searchable vectors, and model identity maintained by indexing jobs.

Request structure

Router validates the HTTP request and obtains the user/database dependencies. **Services** apply rules such as ownership, roles, quota, and readiness. **Repositories/SQLAlchemy** read and update the database. **Celery tasks** perform deferred scan and indexing work.

Why transactions matter

Row locks serialize quota reservations and refresh-token rotation. Arithmetic storage releases avoid lost updates. Idempotent finalization avoids creating duplicate files when upload completion is retried. Alembic tracks reproducible schema changes.

DEDUPLICATION IN ONE SENTENCE

Two logical files can reference one clean physical blob, while retaining separate ownership, permissions, revisions, and logical quota accounting.

11 / DEMO PREPARATION

Run the app and rehearse the demo

Start the existing local project

Terminal / check	Command or address
Project folder	FastAPI_Backend_Project6_CloudVault
Backend services	docker compose up -d --build
Frontend terminal	cd frontend npm ci npm run dev
Workspace / API docs	http://localhost:5173 http://localhost:8000/docs
Health / background logs	http://localhost:8000/health/ready docker compose logs --tail 40 api worker scheduler

For a fresh checkout, prepare private configuration from .env.example first. Keep working SMTP/R2 settings and the local MFA key private. Compose runs migrations when the API starts; preserve database/upload volumes. Initial ClamAV definitions can take several minutes.

A smooth 5-minute demonstration

Time	Action	What to explain
0:00-0:40	Sign in; show folders and quota.	Personal workspace and authenticated access.
0:40-1:40	Upload a small text/PDF; wait for CLEAN.	Validation, quota reservation, background scan.
1:40-2:20	Click the filename; preview and download.	Safe supported previews and signed downloads.
2:20-3:30	Share with a second account as Viewer, then Commenter.	Sender/receiver visibility and role differences.
3:30-4:15	Search for a phrase inside the document.	Extraction/OCR, indexing, ranked snippets.
4:15-5:00	Show Trash/restore and security settings.	Recovery, verification state, optional MFA.

Rehearsal checklist: use two test accounts in separate browser profiles; pre-scan/index a backup sample; verify downloads; check SMTP early; keep recovery codes off the projector; never open private environment files. Prepare a short recording if network-dependent steps fail.

12 / SPEAKING PLAN

An easy 8-minute project presentation

Time	Topic	Suggested wording
0:00-0:45	Problem and objective	"We built CloudVault to make file storage, sharing, and document discovery easier to manage in one workspace."
0:45-1:45	Architecture	"React provides the interface; FastAPI applies the rules. PostgreSQL holds metadata, while local storage or R2 holds the actual bytes. Workers handle expensive processing."
1:45-2:45	Upload lifecycle	"An upload first consumes reserved capacity. It stays unavailable until ClamAV marks it clean, then indexing makes its contents searchable."
2:45-4:45	Live demonstration	Show a prepared file, preview/download, and a second account with a restricted role. Show content search if the index is ready.
4:45-5:45	Security	"Permissions are checked by the API. We use hashed passwords, rotating sessions, expiring verification links, MFA, and rate limits."
5:45-6:45	Engineering decisions	"Background jobs keep requests responsive; row locks protect shared counters; job leases and recovery handle interruptions."
6:45-7:30	Validation and limits	Explain backend and browser coverage, then distinguish optional semantic search and production activation from completed local features.
7:30-8:00	Close	"CloudVault combines safe file access, understandable collaboration, and searchable documents. Our next focus is stronger production validation and richer file support."

Suggested division for a team

Presenter 1: problem, objectives, and architecture. Presenter 2: demo and collaboration. Presenter 3: upload/search implementation. Presenter 4: security, testing, and future work. If there are fewer presenters, combine adjacent parts.

IF YOU FORGET A DETAIL

Return to the lifecycle: authenticate, authorize, upload, scan, index, share, and recover. Explain the purpose of each stage before naming libraries.

13 / QUESTIONS AND ANSWERS

Be ready for the viva

Likely question	A clear, defensible answer
Why FastAPI?	Typed request validation, dependency-based authentication, automatic OpenAPI docs, and a good fit with Python scanning/OCR services.
Why PostgreSQL instead of storing files in the database?	The database is suited to relational metadata and transactions. Object storage/local files hold the bytes, and metadata points to them.
Why Celery and Redis?	Scanning/OCR can be slow. A queue lets the API respond while workers process jobs. Redis also supports atomic rate limits.
Is every upload immediately downloadable?	No. The API requires CLEAN status before allowing normal preview or download.
What if two uploads happen together?	Quota checks use locked user rows and reservations; finalization is retry-safe. Releases use database arithmetic.
How does MFA differ from email verification?	Verification proves access to an inbox. MFA adds a time-based authenticator factor at sign-in. The sender's SMTP app password is unrelated to user MFA.
Can a Viewer be prevented from copying a file?	No. Viewer permits preview/download, and there is no DRM. Shared URLs already issued may work until they expire.
Is the search AI?	Default search is lexical/full-text plus hashed vectors. Learned semantic embeddings are optional and require configuration and reindexing.
Can users search everyone's files?	No. Current content-search queries are restricted to the signed-in owner's eligible files.
Is it production-ready and fully secure?	It has security controls and deployment templates, but public HTTPS rollout, representative load/relevance tests, backup restore drills, and provider-specific checks still need validation.

HANDLING AN UNKNOWN QUESTION

State the boundary honestly: "That is not implemented in the current version. Here is the component we would extend and how we would test it."

14 / EVIDENCE AND ROADMAP

What is verified, and what comes next?

Evidence you can mention

- The broader local implementation previously passed 74 backend unit/integration tests and 8 browser tests. After the latest link/MFA/UI fixes, 10 targeted backend checks and the expanded 10-test browser suite passed. These are separate runs, not additive totals.
- Frontend build and lint passed. The running API was checked for a legacy email-link 303 redirect to port 5173 and working MFA encryption. Earlier checks included live R2 signed download responses and generated OCR samples.
- Backend tests cover permissions, storage/quota, sessions, sharing, workflows, and recovery. Browser tests exercise UI flows with mocked APIs. A complete real-inbox verification click-through remains a separate user check.

Prioritized next improvements

Priority	Improvement	How to validate it
Next	Full two-account, real SMTP/R2 end-to-end tests.	Verify inbox link, preview/download, revoke, expiry, MFA/recovery.
Next	Public HTTPS deployment, monitoring, and restore drills.	Confirm certificates, service isolation, alerting, recovery time.
Then	Representative load and OCR quality evaluation.	Measure API latency, queue delay, scan/OCR throughput, extraction accuracy.
Then	Learned semantic model and relevance evaluation.	Configure matching 384D model, reindex, measure useful search results.
Later	Accessible/mobile polish, richer previews, and shared-search scope.	Keyboard/mobile usability tests; safe format handling; permission-isolation tests.

Source map for preparation

Implementation reference: README.md; IMPLEMENTATION.md; docker-compose.yml; requirements.txt; frontend/package.json; app/api/; app/services/; app/tasks/file_task.py; app/db/models/; frontend/src/; tests/; frontend/e2e/. Setup help: deploy/LOCAL_EMAIL.md and deploy/OPERATIONS.md. Guide prepared from the local source tree; it does not assert that all local changes are published on GitHub.

FINAL MESSAGE

CloudVault demonstrates a complete full-stack workflow: controlled file storage, role-based collaboration, background safety checks, and searchable document content. Present what works, explain why it is designed that way, and be clear about the next validation steps.